

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación

PROGRAMACIÓN I

■ Gestión de Países

Sistema de Administración de Datos con CSV

Trabajo Práctico Integrador

Alumno: Eric Baier

Docente Tutor: Cinthia Rigoni

Año: 2026

Fecha de entrega: Junio 2026

UTN · Facultad Regional · 2026

ÍNDICE DE CONTENIDOS

1.	Marco Teórico	3
1.1	Listas y Diccionarios en Python	3
1.2	Archivos CSV y el módulo csv	4
1.3	Funciones y Modularización	4
2.	Decisiones Técnicas y Arquitectura	5
2.1	Estructura del Programa	5
2.2	Capturas de Pantalla de Ejecución	5
3.	Nota sobre Modularización	8
4.	Dificultades y Conclusiones	9
5.	Bibliografía / Webgrafía	10
6.	Links del Proyecto	10

1. MARCO TEÓRICO

1.1 Listas y Diccionarios en Python

En Python, las **listas** son colecciones ordenadas y mutables que permiten almacenar múltiples valores bajo un único nombre de variable. Se definen entre corchetes y admiten elementos de cualquier tipo. Son la estructura de datos central de este proyecto: cada país se almacena como un elemento dentro de una lista llamada *países*.

Los **diccionarios** (dict) son colecciones de pares clave-valor no ordenados. Permiten acceder a los datos por nombre en lugar de por posición, lo que hace al código más legible y menos propenso a errores de índice. En este proyecto, cada país es un diccionario con las claves: *nombre*, *poblacion*, *superficie* y *continente*.

```
# Ejemplo: lista de diccionarios
países = [
    {"nombre": "Argentina", "poblacion": 45000000, "superficie": 2780400, "continente":
    "America"},
    {"nombre": "Brasil", "poblacion": 215000000, "superficie": 8515767, "continente":
    "America"},
]

# Acceso por clave
print(países[0]["nombre"]) # Argentina
print(países[1]["poblacion"]) # 215000000
```

Figura 1 – Lista de diccionarios representando países

Las listas admiten operaciones como **append()** para agregar elementos, **sorted()** para ordenar, y comprensiones de lista para filtrar. Los diccionarios proveen métodos como **keys()**, **values()** e **items()** para iterar sobre sus componentes.

1.2 Archivos CSV y el módulo csv

CSV (*Comma-Separated Values*) es un formato de texto plano ampliamente utilizado para el intercambio de datos tabulares. Cada fila representa un registro y los campos se separan por comas. Python incluye el módulo estándar **csv** que facilita la lectura y escritura de estos archivos sin necesidad de instalar librerías externas.

```
import csv

# Lectura con DictReader (cada fila es un diccionario)
with open("datos.csv", "r", encoding="utf-8") as f:
    lector = csv.DictReader(f)
    for fila in lector:
        print(fila["nombre"], fila["poblacion"])

# Escritura con DictWriter
with open("datos.csv", "w", newline="", encoding="utf-8") as f:
    campos = ["nombre", "poblacion", "superficie", "continente"]
    escritor = csv.DictWriter(f, fieldnames=campos)
    escritor.writeheader()
    escritor.writerow({"nombre": "Argentina", "poblacion": 45000000, ...})
```

Figura 2 – Lectura y escritura de CSV con el módulo estándar

El parámetro **newline=""** en la apertura del archivo es fundamental al escribir CSV en Windows para evitar la inserción de líneas en blanco adicionales. El encoding **utf-8** garantiza la correcta representación de caracteres especiales como tildes y la letra ñ.

1.3 Funciones y Modularización

Las **funciones** son bloques de código reutilizables definidos con la palabra clave **def**. Permiten dividir un programa en partes más pequeñas, cada una responsable de una tarea específica, lo que facilita la lectura, el mantenimiento y la depuración del código. En este proyecto se definen funciones para cada operación: *cargar_paises()*, *guardar_paises()*, *alta()*, *modificacion()*, *busqueda()*, *filtro()*, *orden()* y *estadisticas()*.

2. DECISIONES TÉCNICAS Y ARQUITECTURA

2.1 Estructura del Programa

El sistema está implementado en un único archivo **main.py** por requerimiento explícito de la cátedra. Se organiza en bloques funcionales claramente delimitados mediante comentarios:

Bloque	Función(es)	Responsabilidad
Validaciones	<code>normalizar_texto()</code> , <code>pedir_texto()</code> , <code>pedir_entero_positivo()</code> , <code>pedir_rango()</code>	Garantizan que el input del usuario sea válido antes de procesar.
Persistencia	<code>cargar_paises()</code> , <code>guardar_paises()</code>	Leen y escriben el archivo <code>datos.csv</code> .
CRUD	<code>visualizar_csv()</code> , <code>alta()</code> , <code>modificacion()</code> , <code>busqueda()</code>	Operaciones básicas sobre los datos.
Análisis	<code>filtro()</code> , <code>orden()</code> , <code>estadisticas()</code>	Permiten explorar y resumir la información.
Interfaz	<code>menu()</code>	Bucle principal que dirige al usuario.

Decisiones de diseño relevantes:

- **Persistencia en CSV simple:** se eligió CSV (en lugar de bases de datos) por su portabilidad, la posibilidad de editarlo con cualquier editor de texto y su alineación con los contenidos de la materia.
- **Validación centralizada:** las funciones `pedir_texto()` y `pedir_entero_positivo()` encapsulan toda la lógica de validación de entrada, evitando código repetido en cada operación.
- **Normalización de texto:** `normalizar_texto()` aplica `.strip().capitalize()` para garantizar consistencia en el almacenamiento (por ejemplo, 'ARGENTINA', 'argentina' y 'Argentina' se guardan siempre como 'Argentina').
- **Continentes predefinidos:** se definió una lista de continentes válidos para evitar inconsistencias en el campo continente del CSV.
- **Escape de errores uniforme:** todas las funciones que retornan `None` permiten al usuario volver al menú sin propagar errores.

2.2 Capturas de Pantalla de Ejecución

Las siguientes capturas muestran la interfaz de consola del sistema durante una sesión típica de uso:

Menú Principal

GESTIÓN DE PAÍSES

- 1 - Visualizacion del listado
- 2 - Alta de país
- 3 - Modificar país
- 4 - Buscar país
- 5 - Filtrar países
- 6 - Ordenar países
- 7 - Estadísticas
- 8 - Salir

Seleccione una opción: _

Alta de País

=== ALTA DE PAÍS ===

Ingrese nombre del país: España

Ingrese población: 47350000

Ingrese superficie: 505990

Ingrese continente: Europa

Datos guardados correctamente.

País agregado correctamente.

Estadísticas

ESTADÍSTICAS

Mayor población: China (1400000000)

Menor población: Islandia (370000)

Promedio de población: 145320000.00

Promedio de superficie: 987400.00

Cantidad de países por continente:

America: 4

Europa: 3

Asia: 2

Africa: 1

Filtro por Continente

FILTROS

- 1 - Continente
- 2 - Rango de población
- 3 - Rango de superficie

Opción: 1

Ingrese continente: Europa

RESULTADOS

Nombre: España

Población: 47350000

Superficie: 505990

Continente: Europa

Figura 3 – Capturas representativas de ejecución: menú principal, alta de país, estadísticas y filtro por continente.

3. NOTA SOBRE MODULARIZACIÓN

■ ■ ■ **Nota académica:** El código completo se desarrolló en un único archivo **main.py** por requerimiento explícito de la cátedra. En un proyecto de mayor escala o en un contexto profesional, la arquitectura recomendada sería dividir el código en módulos separados, como se muestra a continuación.

La **modularización** consiste en separar el código en múltiples archivos .py, cada uno con una responsabilidad específica. Esto mejora la legibilidad, facilita las pruebas unitarias y permite reutilizar componentes en otros proyectos.

Ejemplo de cómo se podría haber estructurado el proyecto con módulos:

```
gestion_paises/  
■ ■ ■ main.py # Solo contiene la llamada a menu()  
■ ■ ■ menu.py # Función menu() y flujo principal  
■ ■ ■ validaciones.py # normalizar_texto(), pedir_texto(), pedir_entero_positivo()  
■ ■ ■ persistencia.py # cargar_paises(), guardar_paises()  
■ ■ ■ operaciones.py # alta(), modificacion(), busqueda(), filtro(), orden(), estadisticas()
```

Estructura de módulos alternativa

Ejemplo concreto: extracción de la función cargar_paises() a su propio módulo:

```
# persistencia.py (módulo separado)  
import csv, os  
ARCHIVO = 'datos.csv'  
  
def cargar_paises():  
    """Lee CSV y devuelve lista."""  
    paises = []  
    if not os.path.exists(ARCHIVO):  
        return paises  
    with open(ARCHIVO, 'r',  
              encoding='utf-8') as f:  
        lector = csv.DictReader(f)  
        for fila in lector:  
            paises.append({  
                'nombre': fila['nombre'],  
                'poblacion': int(  
                    fila['poblacion']),  
                ...  
            })  
    return paises
```

```
# main.py (archivo principal)  
from persistencia import cargar_paises  
from operaciones import alta, filtro  
from menu import menu  
  
# Todo el main queda en 3 líneas:  
if __name__ == '__main__':  
    menu()  
  
# Ventajas:  
# ✓ Cada módulo es testeable  
# de forma independiente.  
# ✓ Si cambia el formato de  
# almacenamiento, solo se  
# modifica persistencia.py.  
# ✓ Código más corto y legible  
# en cada archivo.  
# ✓ Trabajo en equipo más fácil.
```

Figura 4 – Izquierda: módulo persistencia.py separado. Derecha: main.py resultante.

4. DIFICULTADES Y CONCLUSIONES

Dificultades encontradas

■ Manejo de caracteres especiales en CSV

Al guardar datos con tildes o ñ, el archivo se corrompía en algunas configuraciones. La solución fue especificar explícitamente `encoding='utf-8'` tanto en la apertura de lectura como en la escritura, y agregar `newline=""` al abrir el archivo en modo escritura para evitar líneas en blanco duplicadas en Windows.

■ Validación de entrada del usuario

El manejo de inputs inválidos (campos vacíos, texto donde se esperaba un número, números negativos) generó mucho código repetitivo en las primeras versiones. La solución fue centralizar la lógica en las funciones `pedir_texto()` y `pedir_entero_positivo()`, que encapsulan los bucles de validación y permiten al usuario elegir entre reintentar o volver al menú.

■ Consistencia en los datos del CSV

Al cargar datos escritos manualmente en el CSV, aparecían inconsistencias como 'argentina' vs 'Argentina' o 'EUROPA' vs 'Europa'. La función `normalizar_texto()` con `.strip().capitalize()` resolvió este problema al momento de la carga y el alta.

■ Lógica de búsqueda y modificación

La búsqueda y modificación requerían comparaciones case-insensitive para no requerir que el usuario escriba el nombre exactamente igual a como está guardado. Se utilizó `.lower()` en ambos lados de la comparación para garantizar que 'argentina', 'Argentina' y 'ARGENTINA' sean tratados como el mismo país.

Conclusiones

El desarrollo de este Trabajo Práctico Integrador permitió consolidar múltiples conceptos trabajados durante el cuatrimestre. La implementación de un sistema CRUD completo, desde la validación del input hasta la persistencia en archivo, evidenció la importancia de diseñar las funciones con una responsabilidad clara y bien delimitada.

El uso de **listas de diccionarios** como estructura de datos central resultó muy natural para representar colecciones de entidades con múltiples atributos, y el módulo `csv` de la librería estándar demostró ser una herramienta sólida y suficiente para la persistencia de datos de esta escala.

Desde el punto de vista del aprendizaje, el principal aporte fue comprender que la calidad de un programa no se mide solo por si funciona, sino por su capacidad de manejar correctamente los casos borde: input inválido, archivos inexistentes, datos con formato incorrecto. La robustez del código requiere anticipar y gestionar estos escenarios de forma explícita.

5. BIBLIOGRAFÍA / WEBGRAFÍA

[1] Python Software Foundation – Estructuras de Datos: Listas y Diccionarios

Documentación oficial de Python 3 sobre listas, métodos de lista, diccionarios y sus operaciones. Cubre `append()`, `sorted()`, `keys()`, `values()`, comprensiones de lista y casos de uso típicos.

■ <https://docs.python.org/3/tutorial/datastructures.html>

[2] Python Software Foundation – Módulo csv

Referencia completa del módulo csv de la librería estándar. Explica DictReader, DictWriter, los parámetros de apertura de archivos (`newline`, `encoding`) y las consideraciones para distintos sistemas operativos.

■ <https://docs.python.org/3/library/csv.html>

[3] Python Software Foundation – Definición de Funciones

Sección del tutorial oficial que cubre la definición de funciones con `def`, parámetros, valores de retorno, funciones lambda y el concepto de alcance.

■ <https://docs.python.org/3/tutorial/controlflow.html#defining-functions>

[4] Python Software Foundation – Módulo os.path

Documentación del módulo os.path, utilizado en el proyecto para verificar la existencia del archivo CSV antes de intentar leerlo (`os.path.exists`).

■ <https://docs.python.org/3/library/os.path.html>

6. LINKS DEL PROYECTO

■ Repositorio GitHub

<https://github.com/ericbaier774-coder/UTN-TUPaD-Programacion1-TPI>

■ Video Explicativo

<https://youtu.be/WXtZsZ4dIts>